WebRTC Signalling Architecture

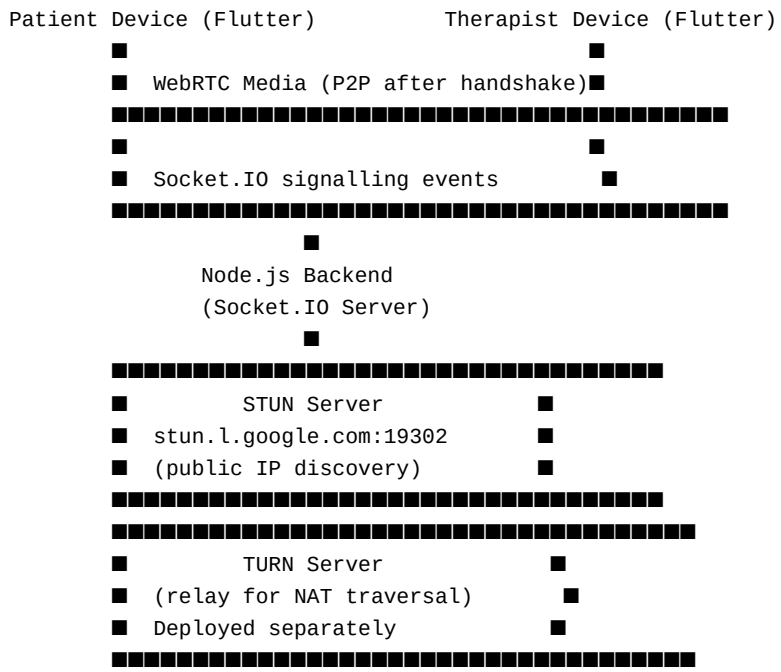
Resilio — Mobile Mental Health Platform for Nepal | Milan Shrestha | ID: 23057135 |
03_Construction/WebRTC_Signalling_Architecture.md

Technical documentation for the WebRTC real-time video and audio consultation feature in Resilio. This document covers the signalling protocol, peer connection lifecycle, Socket.IO event design, and the STUN/TURN configuration. Intended as a technical reference for the implementation described in Chapter 3.

OVERVIEW

Resilio uses **WebRTC** (Web Real-Time Communication) for peer-to-peer video and audio consultations between patients and therapists. WebRTC establishes a direct media channel between the two devices. A **Socket.IO signalling server** (hosted in the Node.js backend) is used to exchange the connection parameters (SDP offers/answers and ICE candidates) that allow the two peers to find and connect to each other.

COMPONENT ARCHITECTURE



GLOSSARY OF WEBRTC TERMS

Term	Definition
SDP (Session Description Protocol)	A format describing media capabilities (codecs, resolution, bandwidth). An "offer" is sent by the caller; an "answer" is sent by the callee
ICE (Interactive Connectivity Establishment)	The process of finding valid network paths (candidates) between two peers
ICE Candidate	A potential network path: a local IP address, a STUN-resolved public IP, or a TURN relay address
STUN (Session Traversal Utilities for NAT)	A server that tells a device its public-facing IP address. Free Google STUN: `stun:stun.l.google.com:19302`
TURN (Traversal Using Relays around NAT)	A relay server that forwards media between peers when a direct connection is impossible (e.g., symmetric NAT on mobile networks)
RTCPeerConnection	The main WebRTC API object that manages the peer connection lifecycle
MediaStream	A stream of audio and/or video tracks from getUserMedia()

Term	Definition
Offer/Answer Model	The two-phase SDP negotiation: caller creates offer → callee creates answer → both set local and remote descriptions

SOCKET.IO SIGNALLING EVENTS

The Node.js Socket.IO server acts as a relay for signalling messages. It does not process the SDP or ICE data — it only routes messages between the two clients in the same room.

Events Emitted by Clients → Server

Event	Payload	Description
`join-room`	`{ roomId, userId }`	Client joins the meeting room identified by the appointment's `meeting_room_id`
`offer`	`{ roomId, offer (SDP) }`	Caller sends SDP offer to the other peer
`answer`	`{ roomId, answer (SDP) }`	Callee sends SDP answer back
`ice-candidate`	`{ roomId, candidate }`	Sends an ICE candidate to the other peer
`leave-room`	`{ roomId }`	Client signals end of session

Events Emitted by Server → Clients

Event	Payload	Description
`user-joined`	`{ userId }`	Notifies the room that a new participant has joined
`offer`	`{ offer (SDP) }`	Forwarded offer from the other client
`answer`	`{ answer (SDP) }`	Forwarded answer from the other client
`ice-candidate`	`{ candidate }`	Forwarded ICE candidate from the other client

Event	Payload	Description
`user-left`	`{ userId }`	Notifies the room that a participant has disconnected

PEER CONNECTION LIFECYCLE

Step-by-Step Signalling Flow

FLUTTER IMPLEMENTATION REFERENCE

RTCPeerConnection Initialisation

```
final Map<String, dynamic> _iceServers = {
  'iceServers': [
    {'urls': 'stun:stun.l.google.com:19302'},
    {
      'urls': 'turn:[TURN_SERVER_IP]:3478',
      'username': '[TURN_USERNAME]',
      'credential': '[TURN_PASSWORD]',
    },
  ]
};

final RTCPeerConnection peerConnection =
    await createPeerConnection(_iceServers);
```

Local Media Stream

```
final Map<String, dynamic> mediaConstraints = {
  'audio': true,
  'video': {
    'facingMode': 'user',
    'width': {'ideal': 1280},
    'height': {'ideal': 720},
  },
};

final MediaStream localStream =
    await navigator.mediaDevices.getUserMedia(mediaConstraints);

localStream.getTracks().forEach((track) {
  peerConnection.addTrack(track, localStream);
});
```

ICE Candidate Handler

```
peerConnection.onIceCandidate = (RTCIceCandidate candidate) {
  if (candidate.candidate != null) {
    socket.emit('ice-candidate', {
      'roomId': roomId,
      'candidate': candidate.toMap(),
    });
  }
};
```

Remote Stream Handler

```
peerConnection.onTrack = (RTCTrackEvent event) {
  if (event.streams.isNotEmpty) {
    setState(() {
      _remoteStream = event.streams.first;
      _remoteRenderer.srcObject = _remoteStream;
    });
  }
};
```

STUN/TURN CONFIGURATION

Why STUN Alone Was Insufficient (Risk R-T01)

During Elaboration phase prototype testing, STUN (using Google's public STUN server) worked correctly on WiFi connections where both devices had distinct public IPs. However, testing on mobile 4G revealed that many mobile network providers use **symmetric NAT** — a NAT type that STUN cannot traverse. In this configuration, the ICE candidate exchange completes but no direct connection can be established.

Solution: A TURN relay server was deployed. The TURN server acts as a media relay — rather than a direct peer-to-peer path, media flows through the TURN server. This is less efficient (higher latency, TURN server bandwidth cost) but universal. The TURN server is only used when a direct path cannot be found.

TURN Server Setup

TURN server was deployed using **Coturn** (open-source TURN server):

```
https://github.com/coturn/coturn
```

Deployed on a cloud VM. Configuration:

- Listening port: 3478 (UDP and TCP)
- TLS port: 5349
- Authentication: long-term credential mechanism (username/password in ICE server config)
- Realm: resilio.app

SESSION ROOM ID DESIGN

Each appointment has a unique `meeting_room_id` generated at booking time:

```
room_{therapistId}_{scheduledTimeMilliseconds}
```

Example: `room_abc123_1710492000000`

Why this format:

- Unique per appointment (therapist + time = unique pair)
- Both patient and therapist derive the same room ID from their appointment record
- No additional room lookup required — the room ID is embedded in the appointment

VIDEO QUALITY AND BANDWIDTH

Connection Type	Approximate Bitrate	Session Quality
WiFi / 4G (strong)	1–2 Mbps video + audio	Excellent — HD video, clear audio
4G (average)	500 Kbps–1 Mbps	Good — reduced resolution, acceptable audio
3G	150–500 Kbps	Fair — low resolution, audio prioritised
2G / Edge	< 150 Kbps	Poor — audio only recommended

WebRTC's **adaptive bitrate** automatically degrades video quality as bandwidth decreases, prioritising audio continuity. Therapists and patients are advised to use WiFi or 4G where possible.

KNOWN LIMITATIONS

Limitation	Detail
One-to-one sessions only	Current Socket.IO room design supports exactly two peers. Group therapy would require mesh networking or SFU (Selective Forwarding Unit) architecture
TURN server bandwidth cost	All mobile-network sessions route through TURN, which has an ongoing bandwidth cost
Session recording not implemented	WebRTC media streams are not recorded; no session replay feature exists
Screen sharing not implemented	Only camera and microphone streams are shared